WEB SEARCH & MINING (WSM) ASSIGNMENT

Information Retrieval System Documentation

A Modular Search Engine with Positional Inverted Indexing, Shunting-Yard Boolean Retrieval, Permuterm Wildcard Processing, and Tkinter GUI

Developer / Author: Saathwik (Saathwik56)

GitHub Repository: github.com/Saathwik56/wsm_search_engine

Tech Stack: Python 3.11 Standard Library (No external dependencies)

GUI Framework: Python Tkinter / ttk

Date: November 2026

Test Coverage: 7 / 7 Automated Unit Tests Passing (100%)

1. Executive Summary

This project implements a complete, self-contained Information Retrieval (IR) system designed for the Web Search and Mining (WSM) curriculum. The search engine features strict modular separation across all seven classical IR stages: document ingestion, tokenization, text normalization, positional inverted index construction, query processing, wildcard processing, and frontend graphical user interface.

The system is built entirely using standard Python without external third-party dependencies (such as NLTK, NumPy, or Lucene), guaranteeing seamless portability and direct execution on standard environments.

> **Key Architectural Highlights**
>
> - **Pure Standard Library:** Employs `dataclasses`, `re`, `unicodedata`, `json`, `math`, and `tkinter`.
> - **Dual Wildcard Resolution:** Implements both a **Permuterm Index** (rotations with binary search prefix lookup) and a **K-gram Index** (trigram postings with regex post-filtering).
> - **Full Boolean Parsing:** Uses Dijkstra's **Shunting-yard algorithm** to convert infix expressions into Reverse Polish Notation (RPN), supporting `AND`, `OR`, `NOT`, nested parentheses, and implicit `AND` conjunctions.
> - **Positional Inverted Index:** Retains exact word positions for phrase queries (e.g., `"inverted index"`) and character offsets for live snippet highlighting.
> - **Tkinter GUI:** Features a search input box with `<Return>` trigger, a distinct Search button, and a read-only scrolled result area with highlighted matches and wildcard diagnostics.

2. Project Architecture & File Layout

The project follows clean software architecture principles with single-responsibility module isolation:

```
wsm_search_engine/
│
├── data/
│   └── sample_corpus/              # 6 IR & WSM text documents for immediate evaluation
│       ├── doc1.txt                # Web search engines & crawling
│       ├── doc2.txt                # Inverted index & tokenization
│       ├── doc3.txt                # Query processing & boolean retrieval
│       ├── doc4.txt                # Wildcard queries & permuterm index
│       ├── doc5.txt                # Web mining & machine learning in search
│       └── doc6.txt                # NLP & text normalization
│
├── src/
│   ├── __init__.py                 # Package initializer
│   ├── document_loader.py          # Module 1: Document Loading & Metadata Management
│   ├── tokenizer.py                # Module 2: Lexical Tokenization & Offset Preservation
│   ├── normalizer.py               # Module 3: Case Folding, Accents, Stopwords & Stemming
│   ├── inverted_index.py           # Module 4: Positional Inverted Index & JSON Persistence
│   ├── query_processor.py          # Module 5: Boolean, Phrase & Ranked Query Processing
│   ├── wildcard_processor.py       # Module 6: Permuterm Index & K-Gram Trigram Index
│   └── gui.py                      # Module 7: Desktop Frontend GUI (Tkinter)
│
```

```
├── tests/
│   ├── __init__.py
│   └── test_search_engine.py          # Automated unit test suite covering Modules 1 through 7
│
├── main.py                            # Application entry point (GUI, CLI, or single-query mode)
├── requirements.txt                   # Standard library dependencies documentation
└── README.md                          # Comprehensive user guide & technical manual
```

3. Detailed Module Specifications

Module 1: Document Loader (`src/document_loader.py`)

The Document Loader is responsible for discovering, reading, and structuring document data from disk or in-memory sources.

- **Document Data Model:** Represents documents via the `Document` dataclass, storing `doc_id` (integer), `title`, absolute `path`, raw `content`, and `metadata`. Provides dynamic properties for `word_count` and `char_count`.
- **Encoding Resilience:** Automatically detects and gracefully handles encoding variants (`utf-8`, `utf-8-sig`, `latin-1`, `cp1252`) with byte replacement fallback to avoid file reading aborts.
- **Directory Ingestion:** Ingests all text files with extensions `.txt`, `.md`, `.data` deterministically sorted for reproducible document identifier assignment.

Module 2: Tokenizer (`src/tokenizer.py`)

Transforms raw character streams into structured token streams while preserving critical spatial metadata:

- **Token Representation:** Encapsulates tokens inside a `Token` dataclass containing:
  - `text` : Normalized lexical string.
  - `position` : Zero-indexed word index in the document ($0, 1, 2, \dots$) essential for positional index construction.
  - `start_char` & `end_char` : Character offsets in the raw document string enabling snippet extraction and exact substring highlighting.
- **Query-Aware Tokenization:** Implements distinct patterns for document indexing (alphanumeric words and contractions) versus query parsing (which preserves wildcard characters `*` and `?`).

Module 3: Text Normalizer & Case Folding (`src/normalizer.py`)

Standardizes vocabulary across documents and queries to eliminate superficial lexical divergence:

- **Case Folding:** Converts all characters to lowercase form to support case-insensitive retrieval.
- **Diacritic & Accent Stripping:** Decomposes Unicode characters via NFKD normalization and eliminates non-spacing mark bytes (e.g. `café` becomes `cafe`).
- **Punctuation Stripping:** Cleans leading/trailing non-alphanumeric punctuation while maintaining valid intra-token characters.
- **Stopword Filtering:** Configurable filtering of 120+ standard English stopwords (e.g. *the, is, at, which*).
- **Porter Stemming:** Self-contained implementation of Martin Porter's 1980 suffix-stripping algorithm (Steps 1a, 1b, 1c, 2, 3, 4, 5a, 5b) allowing grammatical grouping (e.g., *retrieval, retrieving, retrieves* $\rightarrow$ *retriev*).

Module 4: Inverted Index Construction (`src/inverted_index.py`)

Constructs a Positional Inverted Index mapping vocabulary terms to sorted postings records:

- **Posting Structure:** Each `Posting` stores `doc_id`, `term_frequency` (tf), and the sorted array of word `positions` :

```
Posting(doc_id=1, term_frequency=3, positions=[5, 23, 78])
```

- **Index Organization:** Postings lists are guaranteed sorted by `doc_id` ascending, allowing linear-time intersection ($O(M+N)$) during query processing.
- **Persistence:** Provides built-in `save_to_file(path)` and `load_from_file(path)` to serialize and deserialize the entire inverted index to and from JSON format.

Module 5: Query Processing (`src/query_processor.py`)

The core engine supporting Boolean, Phrase, Ranked Free-Text, and Wildcard search modalities:

- **Boolean Retrieval (Shunting-Yard Algorithm):** Parses boolean queries (with operators `AND` , `OR` , `NOT` and parentheses) by converting infix expressions to Reverse Polish Notation (RPN). Automatically inserts implicit `AND` conjunctions between consecutive operands or between operands and `NOT` (e.g. `index NOT permuterm` $\rightarrow$ `index AND NOT permuterm`).
- **Positional Phrase Retrieval:** Evaluates exact word sequences (e.g. `"inverted index"`). The algorithm identifies candidate documents containing all query terms, extracts their position arrays, and verifies whether an unbroken chain of consecutive positions ($pos_{k+1} = pos_k + 1$) exists.
- **Ranked Free-Text Search (TF-IDF):** Computes logarithmic Term Frequency - Inverse Document Frequency with document length normalization:

  ```
  Score(D, Q) = sum( (1 + ln(TF_t,D)) * ln(1 + (N / DF_t)) ) / sqrt(Length_D)
  ```

- **Contextual Snippet Generation:** Computes a localized context window centered around matching terms, snaps boundaries to whole words, and tags matching terms for highlighting.

Module 6: Wildcard Processing (`src/wildcard_processor.py`)

Implements the two fundamental wildcard indexing strategies from classical IR literature:

Technique	Data Structure	Transformation / Query Mechanism	Complexity / Advantage
Permuterm Index	Dictionary of cyclic rotations + sorted prefix array	Appends boundary `$` to term. Rotates query until `*` is at the right edge: • `X*` $\rightarrow$ `$X*` (lookup prefix `$X`) • `*X` $\rightarrow$ `X$*` (lookup prefix `X$`) • `X*Y` $\rightarrow$ `Y$X*` (lookup prefix `Y$X`)	Binary search prefix range lookup $O(\log V)$. Highly efficient for single wildcard queries.
K-gram Index	Inverted index of character 3-grams to vocabulary terms	Pads words with `$term$` . Segments wildcard query into 3-grams, intersects postings lists of candidate terms, and eliminates false positives with regex post-filtering.	Handles multiple arbitrary wildcards (e.g. `p*rm*t*rm`) where rotations are ambiguous.

Module 7: Frontend GUI (`src/gui.py`)

Provides a modern desktop interface developed using Python's native `tkinter` and `ttk` :

- **Search Input Box:** Sizable text entry with automatic focus, Enter key binding (`<Return>`), and a clear button.
- **Search Button:** Primary styled accent button (🔍 `Search`) providing immediate search execution.
- **Read-Only Result Area:** Scrolled text box maintaining `state="disabled"` so users cannot edit results, while allowing full text selection and copying. Utilizes rich Tkinter tags for document headers, metadata badges, scores, wildcard expansions, and highlighted snippet terms.
- **Corpus & Index Management:** Directory browsing, index rebuild trigger, and index status metrics.
- **Pre-processing Toggles:** Live checkboxes to enable or disable Porter Stemming and Stopword Removal.

4. Experimental Verification & Test Results

Automated Unit Test Suite

The project includes a comprehensive test suite in `tests/test_search_engine.py` testing all components end-to-end, including headless GUI initialization.

Test Case	Target Module	Verification Criteria	Status
`test_document_loader`	Module 1	Directory scanning, encoding fallback, metadata generation, and in-memory document injection.	**PASSED**
`test_tokenizer`	Module 2	Word boundary extraction, contraction preservation, sequential token indices, character offsets.	**PASSED**
`test_normalization_and_case_folding`	Module 3	Lowercase folding, accent stripping (café $\rightarrow$ cafe), stopword filtering, and Porter stemmer.	**PASSED**
`test_inverted_index_construction`	Module 4	Postings list creation, positional array verification, term frequency calculation, JSON persistence.	**PASSED**
`test_wildcard_permuterm_and_kgram`	Module 6	Trailing (`sear*`), leading (`*ing`), and middle (`m*n`) wildcards with Permuterm & K-gram.	**PASSED**
`test_query_processing`	Module 5	Boolean AND/OR/NOT evaluation, exact phrase matching (adjacent positions), and ranked search.	**PASSED**
`test_frontend_gui_components`	Module 7	Headless Tkinter widget initialization, read-only state validation, and programmatic search trigger.	**PASSED**

Sample Query Demonstrations

```
# 1. Boolean Query with Conjunction and Negation
Query: "web AND mining NOT machine"
Mode:  BOOLEAN
Hits:  2 documents [doc1.txt, doc5.txt]
Note:  doc5 matches 'web' and 'mining'; documents mentioning 'machine' are excluded.

# 2. Wildcard Query
Query: "sear*"
Mode:  WILDCARD
Expansion: 'sear*' -> ['search', 'searches']
```

```
Hits:  5 documents [doc2.txt, doc1.txt, doc5.txt, doc6.txt, doc3.txt]
Score: Ranked via TF-IDF with sublinear scaling.

# 3. Positional Phrase Query
Query: '"inverted index"'
Mode:  PHRASE
Hits:  3 documents [doc1.txt, doc2.txt, doc3.txt]
Note:  Only documents containing adjacent tokens ('inverted' at pos k, 'index' at pos k+1) matched.
```

5. Execution & User Guide

1. Launch Desktop GUI (Recommended)

```
cd C:\Users\rathi\.gemini\antigravity\scratch\wsm_search_engine
python main.py
```

2. Interactive Command-Line Interface

```
python main.py --cli
```

3. Single-Query Terminal Search

```
python main.py --query "web AND mining"
python main.py --query "sear*"
```

4. Execute Test Suite

```
python -m unittest discover tests
```

> **GitHub Repository**
>
> Source code, documentation, and sample dataset are hosted at:
> **https://github.com/Saathwik56/wsm_search_engine**